SESSION 7 – AADK TASK 7

Building Stateful and Calculative Apps

Name: Sachin
USN: 1JS22CS133
Semester: 8th Semester
Course: Android App Development using Gen AI
Date: 26 February 2026

---

1. Feature Description

The application is a stateful Tip Calculator built using Jetpack Compose. The user enters the bill amount and number of people, selects a tip percentage using a slider, and optionally enables a round-up switch. When the Calculate button is pressed, the app validates inputs, performs calculations, and updates the UI dynamically. This demonstrates structured state management and reactive UI updates in Compose.

---

2. UI Components Used

- OutlinedTextField → Bill Amount input
- OutlinedTextField → Number of People input
- Slider → Tip Percentage selection
- Switch → Round-up option
- Button → Calculate action
- Text → Result display
- Text → Error message display

---

3. State Variables

The following state variables control the UI:

```
var billInput by rememberSaveable { mutableStateOf("") }
var peopleInput by rememberSaveable { mutableStateOf("") }
var tipPercent by rememberSaveable { mutableStateOf(15f) }
var roundUp by rememberSaveable { mutableStateOf(false) }
var result by rememberSaveable { mutableStateOf("") } var
errorMessage by rememberSaveable { mutableStateOf("") }
```

State Table

State Variable	Purpose
billInput	Stores bill amount entered by user
peopleInput	Stores number of people
tipPercent	Stores slider value (tip %)
roundUp	Stores switch state (true/false)
result	Stores calculated per-person amount
errorMessage	Stores validation errors

---

4. Validation Rules

The application performs the following validations:

- Bill amount must be greater than 0.
- Number of people must be at least 1.
- If invalid input is detected:
  - Error message is displayed.
  - Result is cleared.
- If input is valid:
  - Calculation proceeds. ○ Result is displayed dynamically.

---

5. Calculation Logic

1. Convert bill and people inputs into numeric values.
2. Compute tip using:

   ```
   tip = bill × tipPercent / 100
   ```

3. If round-up is enabled: ○ Apply `ceil()` to round the tip.
4. Compute per-person amount:

   ```
   (bill + tip) / numberOfPeople
   ```

5. Format result to two decimal places.
6. Update result state → UI recomposes automatically.

---

6. State Flow Explanation

User Input →

State Variable Update →
Validation Check →
Calculation →
Result State Updated →
Automatic UI Recomposition

Compose automatically updates only the affected UI elements when state changes.

---

7. Handling Configuration Changes

The application uses `rememberSaveable` to preserve state during configuration changes such as screen rotation. This ensures that user input and results are not lost when the device orientation changes.

---

8. Test Cases

Test Case 1 – Valid Input

Bill = 100
People = 4
Tip = 20%
Round Up = false
Expected Result → Correct per-person split displayed.

Test Case 2 – Invalid Bill

Bill = -10
People = 3
Expected → Error message shown, no calculation.

Test Case 3 – Invalid People Count

Bill = 500
People = 0
Expected → Error message displayed.

---

9. Reflection

This task helped me understand how structured state management connects user input, business logic, and UI updates in Jetpack Compose. I learned how validation ensures reliable state transitions and prevents incorrect calculations. Using rememberSaveable improved state persistence during configuration changes. Overall, this exercise strengthened my confidence in building interactive, state-driven applications.

---

10. Conclusion

The Tip Calculator application successfully demonstrates structured data flow, state management, validation, and reactive UI updates in Compose. This task improved understanding of how logic connects directly with UI changes in a modern Android development approach.